

Avoid Enemies and Gaps in 3D platformer Game via DQN

Tsung-Yi Chen
Department of Mathematics
National Central University

June 22, 2025

1 Introduction

Reinforcement learning (RL) has been as a powerful method for training autonomous agents to perform complex tasks through trial and error. Recent progress uses realistic simulation environments to help agents learn faster and try different situations. Unity ML-Agents is a popular toolkit that integrates the Unity game engine with RL algorithms, enabling people to create 3D environments and train agents using state-of-the-art methods [1, 2].

In this project, we employ Unity ML-Agents to train a platformer agent that must navigate a 3D world by avoiding gaps, leaping over stairs, and avoiding enemies that pursuing agent, while reaching the goal. The environment is constructed in Unity, observations are collected via raycasts and relative positions, and a Deep Q-Network (DQN) agent is implemented in Python to learn an optimal policy. Our workflow follows standard setup for Unity ML-Agents and Python dependencies [3].

This report presents the environment design, observation features, network architecture, training process, and experimental results. And then, we will consider some reasons about why the result is not ideal and how to improve the model.

2 Deep Q-Network(DQN)

Deep Q-Network (DQN) is a reinforcement learning algorithm that combines Q-learning with deep neural networks. It was introduced in the paper by Mnih et al. [4] and is one of the first methods to successfully apply deep learning to complex control tasks.

In standard Q-learning, the goal is to learn the Q-value function $Q(s, a)$, which estimates the expected return of taking action a in state s and following the optimal policy afterward. The Q-learning update rule is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

where α is the learning rate and γ is the discount factor.

In DQN, a neural network with parameters θ is used to approximate the Q-function: $Q(s, a; \theta)$. The network is trained to minimize the loss:

$$L(\theta) = \mathbb{E}_{(s,a,r,s')} [(y - Q(s, a; \theta))^2],$$

$$\text{where } y = r + \gamma \max_{a'} Q(s', a'; \theta^-),$$

and θ^- represents the parameters of a separate target network, which is updated less frequently to improve training stability.

To improve learning, DQN uses two key techniques:

- **Experience Replay:** Stores past transitions (s, a, r, s') in a replay buffer and samples mini-batches to break correlation between consecutive steps.
- **Target Network:** A copy of the Q-network with frozen parameters used to compute target values y . It is updated periodically to the latest parameters θ .

This approach helped solve many challenging control problems and became a famous method in deep reinforcement learning.

3 Observation Design

The agent’s ability to navigate the platform world depends critically on its observations. We eventually set a 16-dimensional feature vector that captures all information needed to (1) move toward the goal, (2) detect and avoid gaps, (3) judge when to jump, and (4) dodge enemies.

First, we include two components for goal navigation: the normalized distance and the signed angle between the agent’s forward direction and the goal vector. Formally, if $\mathbf{p}$ and $\mathbf{g}$ denote the agent and goal positions, and $\hat{\mathbf{f}}$ the agent’s forward unit vector, we compute

$$d_{\text{goal}} = \frac{\|\mathbf{g} - \mathbf{p}\|}{D_{\text{max}}}$$

$$\theta_{\text{goal}} = \frac{\text{atan2}((\mathbf{g} - \mathbf{p}) \times \hat{\mathbf{f}}, \hat{\mathbf{f}} \cdot (\mathbf{g} - \mathbf{p}))}{\pi}$$

where D_{max} is the scene’s maximum possible distance.

Next, two self-state features record the agent’s speed and grounded status which is detected by a downward raycast:

$$g = \begin{cases} 1, & \text{if grounded(raycast detected),} \\ 0, & \text{otherwise.} \end{cases}$$

To detect obstacles and gaps we use three types of raycasts (see Figure 1):

- horizontal front (normalized hit distance),
- downward front (gap vs. ground),
- upward front (normalized clearance height).

There are 7 horizontal front raycasts with 20 degrees difference. And 5 downward front raycasts to detect ground. There are 5 upward front raycasts to make sure obstacle is jumpable.

We also add two boolean features:

$$\text{gap} = \begin{cases} 1, & \text{front is gap,} \\ 0, & \text{else,} \end{cases}$$

$$\text{climb} = \begin{cases} 1, & \text{front wall but no ceiling,} \\ 0, & \text{else.} \end{cases}$$

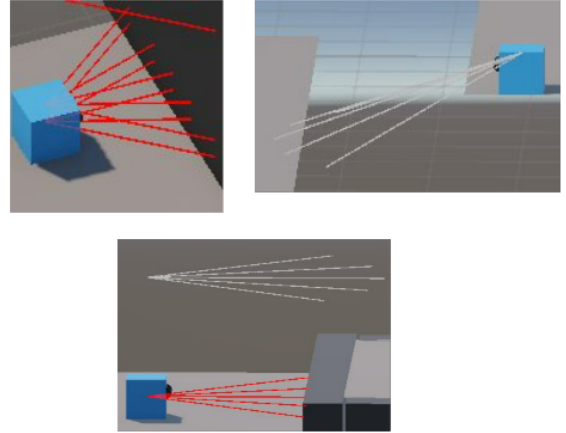


Figure 1: Three types of Ray. Red as detecting wall. Green as detecting ground.

Finally, enemy information is encoded in four features: the normalized distance to the closest enemy, its signed bearing angle, a binary visibility flag, and the normalized relative angle between the enemy direction and the goal direction. These features not only help agent to stay away from the pursuit of enemies but also help agent decide whether it is a good timing to go to the

goal while the enemies are chasing

A summary of the observation vector used by the agent, including spatial, kinematic, and semantic features, is provided in Appendix A.

4 Reward Design

The reward function plays a key role in guiding the agent’s behavior. In this task, the agent receives different types of rewards to encourage progress and avoid failure. The main design principles are as follows:

- **Goal Reaching:** When the agent reaches the goal, it receives a positive reward of +10 and the episode ends.
- **Falling or Collision:** If the agent falls into a pit or touches an enemy, it receives a penalty of −10 and the episode ends.
- **Time Penalty:** A small negative reward is given at every time step to encourage faster solutions.
- **Progress Reward:** The agent receives a small positive reward when its distance to the goal is reduced, and a small penalty if it moves away.
- **Idle or Circling:** A small penalty is applied when the agent stays in place or spins without making forward progress.

This reward design aims to balance safety, efficiency, and goal-directed behavior, helping the agent learn to avoid obstacles and reach the goal effectively.

5 Experiments

We trained the agent using the Deep Q-Network (DQN) algorithm. The training was conducted in Unity using the ML-Agents toolkit. This section describes the key parameters used during training.

Parameter	Value
Learning Rate	0.0005
Batch Size	128
Replay Buffer Size	20000
Discount Factor (γ)	0.995
Target Update Frequency	100
Epsilon Start	1.0
Epsilon End	0.1
Epsilon Decay Rate	0.995

Table 1: DQN training parameters.

5.1 Simple scene for agent and goal

We begin with a simple scenario to verify the basic capabilities of the DQN agent. The environment consists of a square platform surrounded by empty space. The agent spawns at the center, and the goal appears randomly at a distance of 2 to 4 units from the center.

The objective of the agent is to reach the goal without falling off the platform. In this initial setup, no reward is given for approaching the goal; instead, the agent learns to reach the target solely based on its observations, including the direction and distance to the goal.

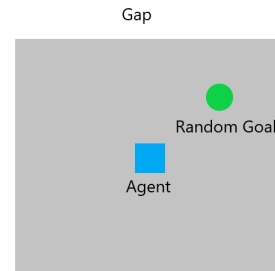


Figure 2: Top-down view of the square platform environment.

Training was conducted using the parameters shown in Table 2. The agent quickly developed survival behavior: by approximately episode 40, it learned not to fall

off the edges. Around episode 60, it started consistently moving toward the goal.

Parameter	Value
Steps per Episode	1000
Number of Episodes	300

Table 2: Training parameters for platform experiment.

To evaluate the learning process, we tracked the loss and the average episodic reward throughout training. As shown in Figure 7, both metrics demonstrate steady improvement over time.

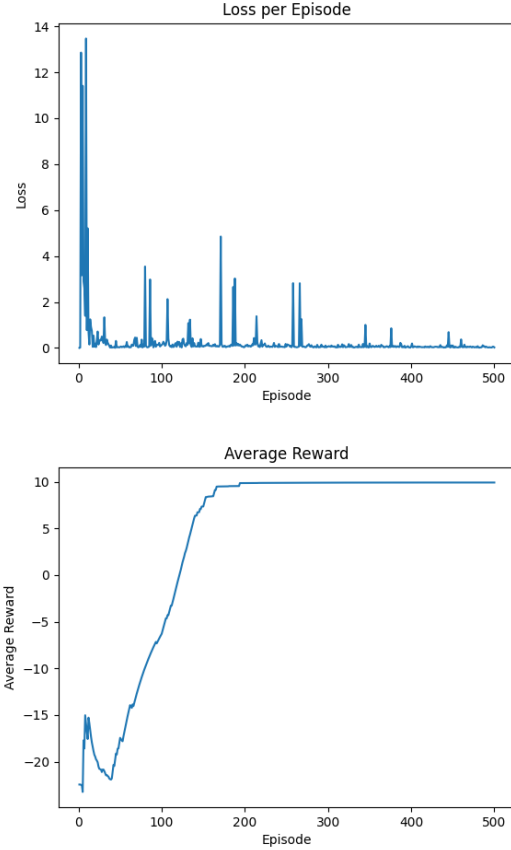


Figure 3: Up: Loss value during training. Down: Average episode reward over time.

5.2 Complex scene for mixed environment

To further challenge the agent, we designed a more complex scenario with vari-

ous obstacles. In the first half of the environment, the agent must carefully avoid gaps and moving enemies. The second half introduces terrain structures such as stairs, walls, and separated platforms that require jumping and climbing.

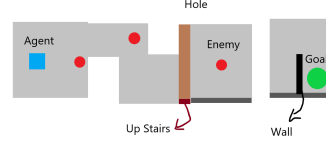


Figure 4: Top-down view of the complex environment used for the second experiment.

The training configuration is shown in Table 3. We increased number of steps per episode to allow sufficient time for exploration in the larger map.

Parameter	Value
Steps per Episode	10000
Number of Episodes	2000

Table 3: Training parameters for the complex scenario.

Although the agent learned to avoid enemies and pits individually in earlier stages, it often exhibited poor global planning during full-environment training. In many episodes, it remained spinning in place, or fell into pits while trying to avoid enemies.

The agent failed to reliably navigate stairs and wall jumps. This was partly due to earlier weaknesses in the wall-detection observations, which lacked sufficient angular and spatial resolution to distinguish wall height or gap width.

The demo could not be completed satisfactorily within the deadline. Nonetheless, the failure cases helped identify key limitations in perception design and reward shaping, offering valuable insights for future improvement.

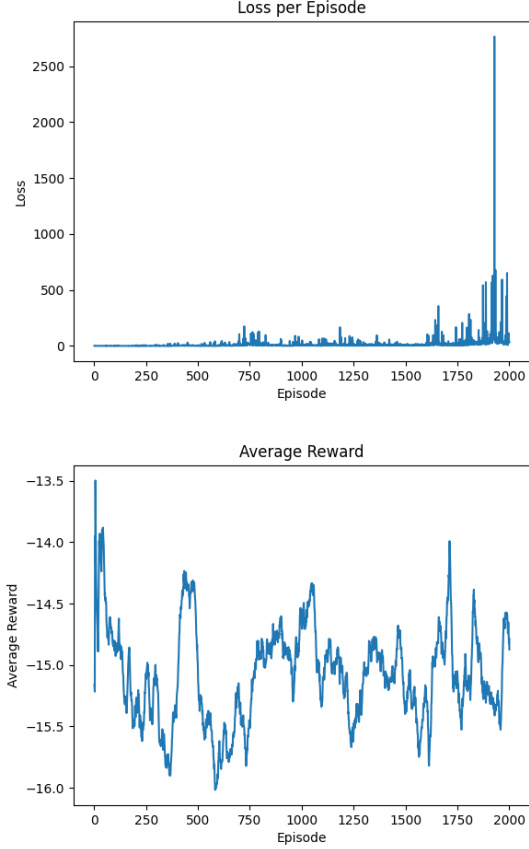


Figure 5: Up: Loss value during training. Down: Average episode reward over time.

5.3 Enhanced Observation and Reward Test

To address the previous limitations, we simplified the environment to focus on dodging enemies and avoiding gaps. We also improved the observation and reward design:

- The angle between the direction to the nearest enemy and the direction to the goal.
- A small penalty is given when the agent contacts a wall.
- A step-wise reward for getting closer to the goal, and penalty for moving away.
- Adjust some reward values and DQN parameters.

The updated training parameters are shown in Table 4.

Parameter	Value
Episodes	1500
Max steps per episode	3000

Table 4: Training parameters used in the enhanced environment.

The map is shown in Figure 6. The agent must avoid enemies and gaps to reach it safely.

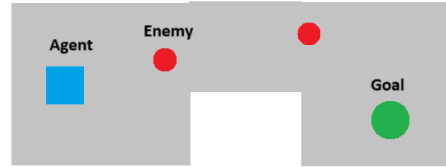
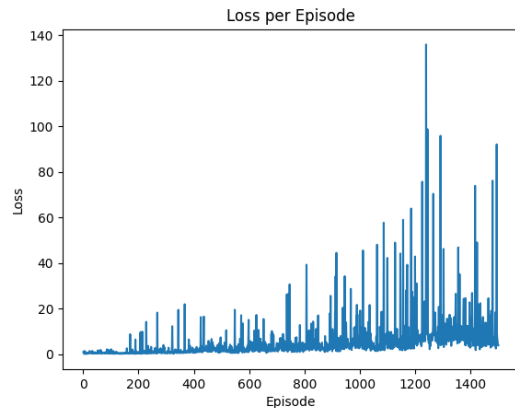


Figure 6: Simplified environment focused on gaps and enemies.

After about 500 episodes, the agent started to learn how to safely reach the goal. Although training was limited and the loss did not fully converge, the improvement was clear compared to earlier experiments.



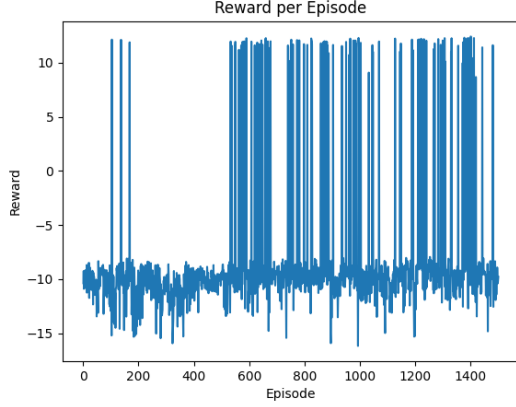


Figure 7: Up: Loss value during training. Down: reward per episode.

We then introduced a staircase and wall section behind the enemies and gaps. In the results, after approximately 1000 episodes, the agent started to learn how to jump over enemies and successfully detect and climb the staircase. Although more training is needed, this setup shows clear progress from the previous attempt.



Figure 8: Simplified environment with stair, enemies, gaps.

6 Conclusion and Future Work

In this work, we developed a reinforcement learning-based agent capable of navigating dynamic environments that include obstacles, gaps, and enemies. Through the design of appropriate reward functions and observation features, the agent was trained using the Deep Q-Network (DQN) algorithm. In simple environments, the agent successfully learned to reach the goal without falling. However, in more complex scenarios involving enemy avoidance and terrain navigation, the agent initially failed to

make consistent progress due to insufficient observations and poor reward shaping.

After revising the observation set to include critical features such as the angle between the enemy and goal directions, and the angular spread of wall-detecting rays, as well as fine-tuning the reward function, the agent demonstrated improved decision-making capabilities. These modifications allowed the agent to avoid enemies more effectively and navigate around hazards. The results indicate that detailed environmental cues are essential for learning effective policies in visually complex environments.

In future work, we plan to extend the observation space with temporal information to help the agent reason across time and make more consistent long-term decisions. We also aim to experiment with other models, such as Proximal Policy Optimization (PPO) models, which is actually used commonly in RL-agents of unity. Additionally, we can design a tons of training plan to make agent learn a skill like jump over the stages without drop from easy to hard environment by model that pretrained last turns. These improvements would contribute to building more robust agents capable of adapting to a broader range of real-world scenarios.

References

- [1] Juliani, A., Berges, V.-P., Vckay, E., Gao, Y., Henry, H., Mattar, M., & Lange, D. (2018). *Unity: A General Platform for Intelligent Agents*. arXiv preprint arXiv:1809.02627.
- [2] Unity-Technologies. *Unity ML-Agents Toolkit — Release 22*. GitHub. Retrieved from: https://github.com/Unity-Technologies/ml-agents/releases/tag/release_22
- [3] LudicWorlds. *ML-Agents Course Conda Setup Guide*. GitHub. Retrieved from: <https://github.com/LudicWorlds/ml-agents-course-assets/blob/main/conda-commands.md>
- [4] Mnih, Volodymyr, et al. "Human-level control through deep reinforcement learning." *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

A Appendix A: Agent Observations

Observation Features Used by the Agent (Total: 16)

1. **Distance to the goal:** Euclidean distance normalized to $[0, 1]$.
2. **Relative angle to goal:** Signed angle between agent's heading and goal, normalized to $[-1, 1]$.
3. **Agent's velocity:** Local velocity magnitude (normalized to estimated maximum).
4. **Ground contact:** Binary flag (1 = grounded, 0 = not).
5. **Horizontal ray - shortest distance:** Among 7 forward-facing rays, minimum hit distance normalized to $[0, 1]$.
6. **Horizontal ray - relative angle:** Signed angle of the closest hit direction, normalized to $[-1, 1]$.
7. **Gap ahead (center):** Binary flag (1 = gap, 0 = ground).
8. **Gap ahead (left):** Same as above, but angled to the left.
9. **Gap ahead (right):** Same as above, but angled to the right.
10. **Jumpable wall ahead:** Binary flag indicating if a front wall is low and clear enough to jump over.
11. **Distance to nearest enemy:** Normalized to $[0, 1]$ within a fixed detection range.
12. **Relative angle to nearest enemy:** Signed, normalized to $[-1, 1]$.
13. **Enemy visibility flag:** 1 if an enemy is detected, otherwise 0.
14. **High-level binary gap forward:** Redundant check to reinforce gap awareness.
15. **High-level climbable flag:** Redundant check for jumpable front walls.
16. **Angle between enemy and goal direction:** Relative signed angle between the vector to enemy and the vector to goal, normalized to $[-1, 1]$.